


UNIVERSITY OF THE PHILIPPINES CEBU
College of Science
Department of Computer Science

TIP: Tracker for Inventory and Profit

A Project Report

In Partial Fulfillment of the Course Requirements
for CMSC 21 – Fundamentals of Programming

Submitted by:

Jason John S. Bisuela
Benedict John D. Dela Cruz
Isabela Cristie B. Ollaban
Kyle Dominic D. Olmedo

Submitted to:

Nasvin F. Del Rosario
Department of Computer Science

Introduction

In today's business climate, efficient profit and loss management and inventory management are crucial for the sustainability of a business especially those who have small retail establishments like minimarts and sari-sari stores. From the study "Accounting Practices of Sari-Sari Stores in Brgy. Makiling, Calamba, Laguna: Basis for Community Extension Program of the College of Business and Accountancy", published in October 2018, the majority of the respondents favored manual methods to bookkeep over computerized ones. The high preference towards the manual accounting is attributed to its perceived lower cost and convenience with the manual methods, especially for the businesses with minimal transactions (Alusen & Javier, 2018).

However, relying on manual calculations for revenue and profit assessments is prone to errors and consumes a lot of time. Since it is time consuming, some of the business owners rarely calculate the profits of their products which could lead to an unawareness of which products sell well and which products don't. Some business owners track the inventory of their products through mental footnotes and paper which could be susceptible to error and could lead to a situation where the product is unavailable when it is in demand, among other problems that arise with poor product tracking. These circumstances could lead SME owners to make poor decisions for their business.

Problem Statement

Some small business owners could have difficulty tracking the inventory of their products since they could use more sophisticated methods which could be prone to error such as writing it in paper or mental footnotes. They also could have a difficulty in checking if their products are making a profit since they may use calculations or in some cases, they won't check for profits and feel if they're making a profit. There is a need for a user-friendly system that could track inventory of items and check for the total revenue, cost, and profit gained from the products to check if the store is making profit or bleeding money. Furthermore, a profit tracking system could lessen the mental load of calculating individual and total expected profits as the system would do it for them.

Solution

Project TIP is a program that aims to make bookkeeping software friendly to sole proprietors that want to start a business with a sari-sari store. Through this, sole proprietors are able to keep track of the sari-sari store's finances, in order to make sound decisions for their business.

Description

With the aid of the accounting cycle, TIP aims to aid the sole proprietor with the entire accounting cycle. The main goal of this project is to help simplify the calculation process for the sole proprietor, with the net profit/loss of the sari-sari store and to help track the inventory of the products currently being sold in the sari-sari store. The calculations of profit/loss can be done erroneously, especially if not done properly, hence this program, alongside its goals to store and track inventory, help keep the numerical data reflective of the store's current financial position. As for the inventory, a periodic inventory system was observed for uniformity.

Initialization

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <stdbool.h>
```

```c
#define MAX_TRANSACTIONS 100
#define MAX_PROD 10
#define PROD_NAME 50
#define CAT_SIZE 20
#define FILENAME_SIZE 1024
#define MAX_LINE 2048
```

The research team utilized the following preprocessor directives for the project. The following #include preprocessor directives was used for the program since there was a need for convenient naming for the inventory management and obtaining of the system time when the transaction was recorded. The following #define preprocessor directives are used for the size of struct arrays and the maximum size of product and category names.

Inventory Management

The code for the product details was derived from the concepts of **struct**, wherein key variables that help describe the product and its current inventory would be used to separate one product from another. The team utilized the following structures to be used for the inventory of products. The product utilizes the struct Product_Details to contain the product's name, weight, price, quantity and costs.

```
struct Product_Details {
  char product_name[PROD_NAME];
  double product_weight;
  char weight_category[CAT_SIZE];
  double price;
  char quantity_category[CAT_SIZE];
  int quantity[2];
  double cost[2];
  char DateAdded[20][2];
};
```

The team accounted for the category of products in the sari-sari store using the struct Categories to separate the products from one another. Each Category contains an array with a size of defined variable MAX_PROD or 10.

```
struct Categories {
  Product_Details Beverage[MAX_PROD];
  Product_Details Canned_Goods[MAX_PROD];
  Product_Details Dairy[MAX_PROD];
  Product_Details Dry_Baking_Goods[MAX_PROD];
  Product_Details Produce[MAX_PROD];
  Product_Details Cleaners[MAX_PROD];
  Product_Details Paper_Goods[MAX_PROD];
  Product_Details Personal_Care[MAX_PROD];
  Product_Details Other[MAX_PROD];
  int BeverageCount;
  int Canned_GoodsCount;
  int DairyCount;
  int Dry_Baking_GoodsCount;
  int ProduceCount;
  int CleanersCount;
  int Paper_GoodsCount;
  int Personal_CareCount;
  int OthersCount;
};
```

The integers BeverageCount to OthersCount function as a limit to ensure that for-loop functions that check the array won't go beyond the index where data is available and to read multiple products in each category.

The main program utilizes the function main_interface() for the user to selectively pick which function of the program that they would want to use and it utilizes aesthetic functions such as TitleScreen(), header() and greeter() so that the program has aesthetic qualities.

```
void main_interface() {
  printf("1. Display Products\n");
  printf("2. Add Products\n");
  printf("3. Modify Product Detail\n");
  printf("4. Remove a Product\n");
  printf("5. Input Sales\n");
  printf("6. Record a Transaction\n");
  printf("7. Print All Transactions\n");
  printf("8. Exit\n");
  printf("Input: ");
}
```

```
}

void TitleScreen() {
  char String[40] = "                ";
  printf("%s////////////    ///     ////////////\n", String);
  printf("%s    ///         ///     ///      //\n", String);
  printf("%s    ///         ///     ///      //\n", String);
  printf("%s    ///         ///     ////////\n", String);
  printf("%s    ///         ///     ///      \n", String);
  printf("%s    ///         ///     ///      \n", String);
}

void header() {
  int len = strlen("  ////        //////   ///      ");
  for (int i = 0; i < 2 * len; i++) {
    printf("-");
  }
}

void greeter() {
  for (int i = 0; i < 25; i++)
    printf(" ");
  printf("WELCOME USER!");
  for (int i = 0; i < 30; i++)
    printf(" ");
}
```

The inventory side of the program utilizes a text file named “product.txt” to check on the items that were initialized or inputted by the user. It has the format of:

Category | Product_Name | Price | Weight_Category | Weight_of_Product | Quantity_Category | Quantity of object

```
Beverage|Coke Sakto|100.00|grams|20.00|pcs|30
Canned Goods|Century Tuna Spicy|200.00|grams|30.00|pcs|30
```

```c
void InitializeItem(Categories *categories) {
  FILE *file = fopen("products.txt", "r");
  if (!file) {
    perror("Failed to open file");
    exit(1);
  }

  char line[2048];
  while (fgets(line, sizeof(line), file)) {
    Product_Details product;
    char category[50];
    sscanf(line, "%[^|]|%[^|]|%lf|%[^|]|%lf|%[^|]|%d", category,
           product.product_name, &product.price, product.weight_category,
           &product.product_weight, product.quantity_category,
           &product.quantity[0]);
```

Afterwards, the function InitializeItem() reads from the file and checks for the products in the file and assigns them into the console. The function splits the line into multiple data using sscanf and the first data Category is used to determine if the item in the text file belongs to a certain category. If the item in the text file belongs to a certain category, the category count from the struct Categories will be used as an index so that the item and its product details will be assigned as a part of that category's products.

```
    if (strcmp(category, "Beverage") == 0) {
      categories->Beverage[categories->BeverageCount++] = product;
    } else if (strcmp(category, "Canned_Goods") == 0) {
      categories->Canned_Goods[categories->Canned_GoodsCount++] = product;
    } else if (strcmp(category, "Dairy") == 0) {
      categories->Dairy[categories->DairyCount++] = product;
    } else if (strcmp(category, "Dry_Baking_Goods") == 0) {
      categories->Dry_Baking_Goods[categories->Dry_Baking_GoodsCount++] =
          product;
    } else if (strcmp(category, "Produce") == 0) {
      categories->Produce[categories->ProduceCount++] = product;
    } else if (strcmp(category, "Cleaners") == 0) {
      categories->Cleaners[categories->CleanersCount++] = product;
    } else if (strcmp(category, "Paper_Goods") == 0) {
      categories->Paper_Goods[categories->Paper_GoodsCount++] = product;
    } else if (strcmp(category, "Personal_Care") == 0) {
      categories->Personal_Care[categories->Personal_CareCount++] = product;
    } else if (strcmp(category, "Other") == 0) {
      categories->Other[categories->OthersCount++] = product;
    }
  }
```

After the product is initialized, there are four functions in the program that will use the inventory: Display Products, Add Products, Remove Products, and Modify Product Details. These functions will use struct Categories. In the main function, the research team initialized an instance of the struct Categories Item_Store and a struct pointer ItemP, ItemP was then assigned to the address of Item_Store. The four inventory functions use ItemP or a struct pointer of Categories as a pointer.

```
  int transaction_count = 0;
  Categories Item_Store[100], *ItemP;
  ItemP = Item_Store;
  InitializeItem(ItemP);
  header();
  printf("\n");
  TitleScreen();
  header();
  printf("\n\n");
  greeter();
  printf("\n");
  int loop = 1;
```

```
case 1:
  Display_Interface(ItemP);
  break;
case 2:
  Add_Product(ItemP);
  break;
case 3:
  ModifyProduct(ItemP);
  break;
case 4:
  RemoveProduct(ItemP);
  break;
case 5:
```

For the display function, it utilizes for-loops and the struct pointer of Categories. The Display function consists of the DisplayInterface(), the multiple display functions involving the different categories, and the DisplayAll() function which outputs every item inside the inventory. The Display Interface follows a similar format from the main interface except the choices involve displaying the categories and the option of displaying all items.

```
void Display_Interface(Categories *Category) {
  int loop = 1, choice;
  while (loop) {
    printf("\nDisplay Interface\n");
    printf("1. Beverages\n");
    printf("2. Canned Goods\n");
    printf("3. Dairy\n");
    printf("4. Dry Baking Goods\n");
    printf("5. Produce\n");
    printf("6. Cleaners\n");
    printf("7. Paper Goods\n");
    printf("8. Personal Care\n");
    printf("9. Others\n");
    printf("10. All\n");
    printf("11. Exit\n");
    printf("Input: ");
    scanf("%i", &choice);
    switch (choice) {
    case 1:
```

The DisplayInterface() is inside a loop to ensure that if the user wants to display another category, they don't need to input back from the starting menu. The different display functions are inside the switch cases.

```
switch (choice) {
case 1:
  DisplayBeverage(Category);
  break;
case 2:
  DisplayCanned_Goods(Category);
  break;
case 3:
  DisplayDairy(Category);
  break;
case 4:
  DisplayDry_Baking_Goods(Category);
  break;
case 5:
  DisplayProduce(Category);
  break;
case 6:
  DisplayCleaners(Category);
  break;
case 7:
  DisplayPaper_Goods(Category);
  break;
case 8:
  DisplayPersonal_Care(Category);
  break;
case 9:
  DisplayOthers(Category);
  break;
case 10:
  DisplayAll(Category);
  break;
case 11:
  loop = 0;
  break;
default:
```

The Display Category functions are similar to one another in that they use the same process in outputting everything inside the category array. They utilize the category count to function as a limit for the loops to not go far when displaying the product and its details.

```
  }
}

void DisplayBeverage(Categories *Categories) {
  printf("\nBeverages\n");
  for (int i = 0; i < Categories->BeverageCount; i++) {
    printf("\nItem %i\n", i + 1);
    printf("%s\n", Categories->Beverage[i].product_name);
    printf("Price: %.2f\n", Categories->Beverage[i].price);
    printf("Quantity: %i\n", Categories->Beverage[i].quantity[0]);
  }
}

void DisplayCanned_Goods(Categories *Categories) {
```

```
void DisplayCanned_Goods(Categories *Categories) {
  printf("\nCanned Goods\n");
  for (int i = 0; i < Categories->Canned_GoodsCount; i++) {
    printf("\nItem %i\n", i + 1);
    printf("%s\n", Categories->Canned_Goods[i].product_name);
    printf("Price: %.2f\n", Categories->Canned_Goods[i].price);
    printf("Quantity: %i\n", Categories->Canned_Goods[i].quantity[0]);
  }
}
```

```
void DisplayDairy(Categories *Categories) {
  printf("\nDairy\n");
  for (int i = 0; i < Categories->DairyCount; i++) {
    printf("\nItem %i\n", i + 1);
    printf("%s\n", Categories->Dairy[i].product_name);
    printf("Price: %.2f\n", Categories->Dairy[i].price);
    printf("Quantity: %i\n", Categories->Dairy[i].quantity[0]);
  }
}
```

```
}

void DisplayDry_Baking_Goods(Categories *Categories) {
  printf("\nDry Baking Goods\n");
  for (int i = 0; i < Categories->Dry_Baking_GoodsCount; i++) {
    printf("\nItem %i\n", i + 1);
    printf("%s\n", Categories->Dry_Baking_Goods[i].product_name);
    printf("Price: %.2f\n", Categories->Dry_Baking_Goods[i].price);
    printf("Quantity: %i\n", Categories->Dry_Baking_Goods[i].quantity[0]);
  }
}

void DisplayProduce(Categories *Categories) {
  printf("\nProduce\n");
  for (int i = 0; i < Categories->ProduceCount; i++) {
    printf("\nItem %i\n", i + 1);
    printf("%s\n", Categories->Produce[i].product_name);
    printf("Price: %.2f\n", Categories->Produce[i].price);
    printf("Quantity: %i\n", Categories->Produce[i].quantity[0]);
  }
}

void DisplayCleaners(Categories *Categories) {
  printf("\nCleaners\n");
  for (int i = 0; i < Categories->CleanersCount; i++) {
    printf("\nItem %i\n", i + 1);
    printf("%s\n", Categories->Cleaners[i].product_name);
    printf("Price: %.2f\n", Categories->Cleaners[i].price);
    printf("Quantity: %i\n", Categories->Cleaners[i].quantity[0]);
  }
}

void DisplayPaper_Goods(Categories *Categories) {
  printf("\nPaper Goods\n");
  for (int i = 0; i < Categories->Paper_GoodsCount; i++) {
    printf("\nItem %i\n", i + 1);
    printf("%s\n", Categories->Paper_Goods[i].product_name);
    printf("Price: %.2f\n", Categories->Paper_Goods[i].price);
    printf("Quantity: %i\n", Categories->Paper_Goods[i].quantity[0]);
  }
}
```

```
void DisplayPersonal_Care(Categories *Categories) {
  printf("\nPersonal Care\n");
  for (int i = 0; i < Categories->Personal_CareCount; i++) {
    printf("\nItem %i\n", i + 1);
    printf("%s\n", Categories->Personal_Care[i].product_name);
    printf("Price: %.2f\n", Categories->Personal_Care[i].price);
    printf("Quantity: %i\n", Categories->Personal_Care[i].quantity[0]);
  }
}

void DisplayOthers(Categories *Categories) {
  printf("\nOthers\n");
  for (int i = 0; i < Categories->OthersCount; i++) {
    printf("\nItem %i\n", i + 1);
    printf("%s\n", Categories->Other[i].product_name);
    printf("Price: %.2f\n", Categories->Other[i].price);
    printf("Quantity: %i\n", Categories->Other[i].quantity[0]);
  }
}
```

The DisplayAll() function basically outputs all of the other display functions in a certain order.

```
void DisplayAll(Categories *Categories) {
  DisplayBeverage(Categories);
  DisplayCanned_Goods(Categories);
  DisplayDairy(Categories);
  DisplayDry_Baking_Goods(Categories);
  DisplayProduce(Categories);
  DisplayCleaners(Categories);
  DisplayPaper_Goods(Categories);
  DisplayPersonal_Care(Categories);
  DisplayOthers(Categories);
}
```

Like the display function, the Add products function uses the struct pointer for Categories and has an interface inquiring the user about the category of the product that the user would want to add.

```
void Add_Product(Categories *Category) {
  int i = 0;
  int option, option2;
  Product_Details new_product;
  char category_name[20];

  printf("\nSelect a Category for the New Product\n");
  printf("1. Beverages\n");
  printf("2. Canned Goods\n");
  printf("3. Dairy\n");
  printf("4. Dry Baking Goods\n");
  printf("5. Produce\n");
  printf("6. Cleaners\n");
  printf("7. Paper Goods\n");
  printf("8. Personal Care\n");
  printf("9. Others\n");

  printf("Option: ");
  scanf("%d", &option);
  option2 = option;

  switch (option) {
  case 1:
    for (; i < MAX_PROD && strlen(Category->Beverage[i].p
```

The switch cases are used to check the index where the category array is empty so that it will be where the new product is added.

```
option2 = option;

switch (option) {
case 1:
  for (; i < MAX_PROD && strlen(Category->Beverage[i].product_name) != 0; i++)
    ; // Find the index where the product name is empty
  strcpy(category_name, "Beverage");
  break;
case 2:
  for (; i < MAX_PROD && strlen(Category->Canned_Goods[i].product_name) != 0;
       i++)
    ;
  strcpy(category_name, "Canned Goods");
  break;
case 3:
  for (; i < MAX_PROD && strlen(Category->Dairy[i].product_name) != 0; i++)
    ;
  strcpy(category_name, "Dairy");
  break;
case 4:
  for (; i < MAX_PROD &&
         strlen(Category->Dry_Baking_Goods[i].product_name) != 0;
       i++)
    ;
  strcpy(category_name, "Dry Baking Goods");
  break;
case 5:
  for (; i < MAX_PROD && strlen(Category->Produce[i].product_name) != 0; i++)
    ;
  strcpy(category_name, "Produce");
  break;
case 6:
  for (; i < MAX_PROD && strlen(Category->Cleaners[i].product_name) != 0; i++)
    ;
  strcpy(category_name, "Cleaners");
  break;
case 7:
  for (; i < MAX_PROD && strlen(Category->Paper_Goods[i].product_name) != 0;
       i++)
    ;
  strcpy(category_name, "Paper Goods");
  break;
case 8:
  for (; i < MAX_PROD && strlen(Category->Personal_Care[i].product_name) != 0;
       i++)
    ;
  strcpy(category_name, "Personal Care");
  break;
```

```
  break;
case 7:
  for (; i < MAX_PROD && strlen(Category->Paper_Goods[i].product_name) != 0;
       i++)
    ;
  strcpy(category_name, "Paper Goods");
  break;
case 8:
  for (; i < MAX_PROD && strlen(Category->Personal_Care[i].product_name) != 0;
       i++)
    ;
  strcpy(category_name, "Personal Care");
  break;
case 9:
  for (; i < MAX_PROD && strlen(Category->Other[i].product_name) != 0; i++)
    ;
  strcpy(category_name, "Others");
  break;
}
```

```
}

if (i < MAX_PROD) {
  printf("Enter New Product Name: ");
  scanf(" %[^\n]s", new_product.product_name);
  printf("Enter Product Price: ");
  scanf("%lf", &new_product.price);
  printf("Enter Product Weight: ");
  scanf("%lf", &new_product.product_weight);
  printf("Enter Weight Category: ");
  scanf(" %[^\n]s", new_product.weight_category);
  printf("Enter Product Quantity: ");
  scanf("%d", &new_product.quantity[0]);
  printf("Enter Quantity Category: ");
  scanf(" %[^\n]s", new_product.quantity_category);
```

The program asks for details about the product such as its name, price, weight, weight category, quantity available, and quantity category. After the user inputs the details, the details will then be saved to the product.txt file through the use of file handling and appending the new product's data.

```
//Register Purchases transaction
double amount;
double cost;
char small_note[20];
amount = cost * new_product.quantity[0]; //to be added ang cost function
NoteExplainer(small_note, new_product.quantity[0], new_product.product_name, "Purchases");
RecordTransaction(notebook, transaction_count, "Purchases", "Cash", amount, small_note);

// Update "products.txt" file with the new product details
FILE *file = fopen("products.txt", "a");
if (file != NULL) {
  fprintf(file, "%s|%s|%.2f|%s|%.2f|%s|%d\n", category_name,
          new_product.product_name, new_product.price,
          new_product.weight_category, new_product.product_weight,
          new_product.quantity_category, new_product.quantity[0]);
  fclose(file);
} else {
  printf("Error opening file.\n");
}
```

After the details are saved onto the file, the Product Detail struct new_product's details are then assigned to the Category of the product at the index where the product name is empty.

```
    // Update the appropriate category array within the Categories struct with
    // the new product
    switch (option2) {
    case 1:
      Category->Beverage[i] = new_product;
      break;
    case 2:
      Category->Canned_Goods[i] = new_product;
      break;
    case 3:
      Category->Dairy[i] = new_product;
      break;
    case 4:
      Category->Dry_Baking_Goods[i] = new_product;
      break;
    case 5:
      Category->Produce[i] = new_product;
      break;
    case 6:
      Category->Cleaners[i] = new_product;
      break;
    case 7:
      Category->Paper_Goods[i] = new_product;
      break;
    case 8:
      Category->Personal_Care[i] = new_product;
      break;
    case 9:
      Category->Other[i] = new_product;
      break;
    }
    printf("New product added successfully.\n");
  } else {
    printf("Category is full. Cannot add new product.\n");
  }
}
```

The Modify function uses the same parameters and interface from the Add Product function. After the user selects the category of the product they want to modify, they must input the name of their product so that it will be searched through the array of the category.

```c
void ModifyProduct(Categories *Category) {
  int option, category_choice, index;
  char category_name[20];
  char product_name[PROD_NAME];

  printf("\nSelect a Category to Modify the Product\n");
  printf("1. Beverages\n");
  printf("2. Canned Goods\n");
  printf("3. Dairy\n");
  printf("4. Dry Baking Goods\n");
  printf("5. Produce\n");
  printf("6. Cleaners\n");
  printf("7. Paper Goods\n");
  printf("8. Personal Care\n");
  printf("9. Others\n");

  printf("Option: ");
  scanf("%d", &category_choice);

  switch (category_choice) {
  case 1:
    strcpy(category_name, "Beverage");
    break;
  case 2:
    strcpy(category_name, "Canned Goods");
    break;
  case 3:
    strcpy(category_name, "Dairy");
    break;
  case 4:
    strcpy(category_name, "Dry Baking Goods");
    break;
  case 5:
    strcpy(category_name, "Produce");
    break;
  case 6:
    strcpy(category_name, "Cleaners");
    break;
  case 7:
    strcpy(category_name, "Paper Goods");
    break;
  case 8:
    strcpy(category_name, "Personal Care");
    break;
  case 9:
    strcpy(category_name, "Others");
    break;
```

If the Modify function locates the product at the certain category, it will move on to the next interface which asks the user what detail they want to change.

```
}

printf("Enter the Product Name to Modify: ");
scanf(" %[^\n]s", product_name);

switch (category_choice) {
case 1:
  index = SearchProduct(Category->Beverage, Category->BeverageCount,
                        product_name);
  break;
case 2:
  index = SearchProduct(Category->Canned_Goods, Category->Canned_GoodsCount,
                        product_name);
  break;
case 3:
  index = SearchProduct(Category->Dairy, Category->DairyCount, product_name);
  break;
case 4:
  index = SearchProduct(Category->Dry_Baking_Goods,
                        Category->Dry_Baking_GoodsCount, product_name);
  break;
case 5:
  index =
      SearchProduct(Category->Produce, Category->ProduceCount, product_name);
  break;
case 6:
  index = SearchProduct(Category->Cleaners, Category->CleanersCount,
                        product_name);
  break;
case 7:
  index = SearchProduct(Category->Paper_Goods, Category->Paper_GoodsCount,
                        product_name);
  break;
case 8:
  index = SearchProduct(Category->Personal_Care, Category->Personal_CareCount,
                        product_name);
  break;
case 9:
  index = SearchProduct(Category->Other, Category->OthersCount, product_name);
  break;
default:
  printf("Invalid option.\n");
  return;
}

if (index == -1) {
  printf("Product not found.\n");
  return;
}
```

```c
Product_Details *product;
switch (category_choice) {
case 1:
  product = &Category->Beverage[index];
  break;
case 2:
  product = &Category->Canned_Goods[index];
  break;
case 3:
  product = &Category->Dairy[index];
  break;
case 4:
  product = &Category->Dry_Baking_Goods[index];
  break;
case 5:
  product = &Category->Produce[index];
  break;
case 6:
  product = &Category->Cleaners[index];
  break;
case 7:
  product = &Category->Paper_Goods[index];
  break;
case 8:
  product = &Category->Personal_Care[index];
  break;
case 9:
  product = &Category->Other[index];
  break;
}

printf("Select the Detail to Modify:\n");
printf("1. Product Name\n");
printf("2. Price\n");
printf("3. Product Weight\n");
printf("4. Weight Category\n");
printf("5. Quantity\n");
printf("6. Quantity Category\n");
printf("Option: ");
scanf("%d", &option);
```

The program asks the user what product detail they want to change. After the user inputs their desired product detail to be changed, the product detail changes will be updated in the product.txt where the line containing the product name will be rewritten so that the product changes could be saved outside of the console. It utilizes file handling so that the code could be rewritten, the product's details including the updated details are updated in the text file.

```c
    // Update the file with the modified details
    FILE *file = fopen("products.txt", "r+");
    if (file == NULL) {
        printf("Error opening file.\n");
        return;
    }

    char line[2048];
    long pos;
    while (fgets(line, sizeof(line), file)) {
        pos = ftell(file);
        char file_category[50], file_name[PROD_NAME];
        sscanf(line, "%[^|]|%[^|]", file_category, file_name);
        if (strcmp(file_category, category_name) == 0 &&
            strcmp(file_name, product_name) == 0) {
            fseek(file, pos - strlen(line) - 1, SEEK_SET);
            fprintf(file, "%s|%s|%.2f|%s|%.2f|%s|%d\n", category_name,
                    product->product_name, product->price, product->weight_category,
                    product->product_weight, product->quantity_category,
                    product->quantity[0]);
            break;
        }
    }

    fclose(file);
}
```

The Remove Product function is similar to the Modifier functions where they first inquire about the category of the product they seek to delete.

```c
void RemoveProduct(Categories *Category) {
    FILE *file, *temp;
    int category_choice;
    char category_name[50];
    char filename[] = "products.txt";
    char temp_filename[] = "temp.txt";
    char target_product[100];

    printf("\nSelect a Category to Remove Product\n");
    printf("1. Beverages\n");
    printf("2. Canned Goods\n");
    printf("3. Dairy\n");
    printf("4. Dry Baking Goods\n");
    printf("5. Produce\n");
    printf("6. Cleaners\n");
    printf("7. Paper Goods\n");
    printf("8. Personal Care\n");
    printf("9. Others\n");
```

After the user selects the category of the product, they are asked to input the name of the product they are going to delete.

```c
switch (category_choice) {
    case 1:
        strcpy(category_name, "Beverage");
        break;
    case 2:
        strcpy(category_name, "Canned_Goods");
        break;
    case 3:
        strcpy(category_name, "Dairy");
        break;
    case 4:
        strcpy(category_name, "Dry_Baking_Goods");
        break;
    case 5:
        strcpy(category_name, "Produce");
        break;
    case 6:
        strcpy(category_name, "Cleaners");
        break;
    case 7:
        strcpy(category_name, "Paper_Goods");
        break;
    case 8:
        strcpy(category_name, "Personal_Care");
        break;
    case 9:
        strcpy(category_name, "Other");
        break;
    default:
        printf("Invalid option.\n");
        return;
}

printf("Enter the Product Name to Remove: ");
scanf(" %[^\n]", target_product);
```

After the product name was inputted, the program updates the product.txt file to not include the product.

```
printf("Enter the Product Name to Remove: ");
scanf(" %[^\n]", target_product);

file = fopen(filename, "r");
temp = fopen(temp_filename, "w");

if (file == NULL || temp == NULL) {
    printf("Error opening file(s)\n");
    return;
}

bool product_found = false;
char line[2048];

while (fgets(line, sizeof(line), file) != NULL) {
    char current_category[50];
    char product_name[100]; // Assuming product names won't exceed 100 characters

    // Parse the line to get the category and product name
    sscanf(line, "%[^|]|%[^|]", current_category, product_name);

    if (strcmp(current_category, category_name) == 0 && strcmp(product_name, target_product) == 0) {
        product_found = true;
        continue; // Skip writing this product to temp file
    }
    fputs(line, temp);
}

fclose(file);
fclose(temp);
```

The program creates a duplicate of the product.txt file known as temp.txt where temp.txt copies everything from product.txt except for the specific product that was removed. Temp.txt's data is then assigned to product.txt, updating the program to not include the product to be removed.

```c
if (!product_found) {
    printf("Product \"%s\" not found in category: %s\n", target_product, category_name);
    remove(temp_filename); // Remove the temporary file if product not found
} else {
    remove(filename);
    rename(temp_filename, filename);
    printf("Product \"%s\" removed successfully\n", target_product);

    // Update the corresponding category array in memory
    switch (category_choice) {
        case 1:
            for (int i = 0; i < Category->BeverageCount; i++) {
                if (strcmp(Category->Beverage[i].product_name, target_product) == 0) {
                    for (int j = i; j < Category->BeverageCount - 1; j++) {
                        Category->Beverage[j] = Category->Beverage[j + 1];
                    }
                    Category->BeverageCount--;
                    break;
                }
            }
            break;
        case 2:
            for (int i = 0; i < Category->Canned_GoodsCount; i++) {
                if (strcmp(Category->Canned_Goods[i].product_name, target_product) == 0) {
                    for (int j = i; j < Category->Canned_GoodsCount - 1; j++) {
                        Category->Canned_Goods[j] = Category->Canned_Goods[j + 1];
                    }
                    Category->Canned_GoodsCount--;
                    break;
                }
            }
            break;
        case 3:
            for (int i = 0; i < Category->DairyCount; i++) {
                if (strcmp(Category->Dairy[i].product_name, target_product) == 0) {
                    for (int j = i; j < Category->DairyCount - 1; j++) {
                        Category->Dairy[j] = Category->Dairy[j + 1];
                    }
                    Category->DairyCount--;
                    break;
                }
            }
```

If the product is not found, then the program will output the message: “Product %s not found in category: %s” where %s is product_name and category respectively. If the product was successfully removed from product.txt, the current array where it was previously used is updated to not include the product.

```
    break;
case 5:
    for (int i = 0; i < Category->ProduceCount; i++) {
        if (strcmp(Category->Produce[i].product_name, target_product) == 0) {
            for (int j = i; j < Category->ProduceCount - 1; j++) {
                Category->Produce[j] = Category->Produce[j + 1];
            }
            Category->ProduceCount--;
            break;
        }
    }
    break;
case 6:
    for (int i = 0; i < Category->CleanersCount; i++) {
        if (strcmp(Category->Cleaners[i].product_name, target_product) == 0) {
            for (int j = i; j < Category->CleanersCount - 1; j++) {
                Category->Cleaners[j] = Category->Cleaners[j + 1];
            }
            Category->CleanersCount--;
            break;
        }
    }
    break;
case 7:
    for (int i = 0; i < Category->Paper_GoodsCount; i++) {
        if (strcmp(Category->Paper_Goods[i].product_name, target_product) == 0) {
            for (int j = i; j < Category->Paper_GoodsCount - 1; j++) {
                Category->Paper_Goods[j] = Category->Paper_Goods[j + 1];
            }
            Category->Paper_GoodsCount--;
            break;
        }
    }
    break;
case 8:
    for (int i = 0; i < Category->Personal_CareCount; i++) {
        if (strcmp(Category->Personal_Care[i].product_name, target_product) == 0) {
            for (int j = i; j < Category->Personal_CareCount - 1; j++) {
                Category->Personal_Care[j] = Category->Personal_Care[j + 1];
            }
            Category->Personal_CareCount--;
            break;
        }
    }
    break;
case 9:
    for (int i = 0; i < Category->OthersCount; i++) {
        if (strcmp(Category->Other[i].product_name, target_product) == 0) {
            for (int j = i; j < Category->OthersCount - 1; j++) {
                Category->Other[j] = Category->Other[j + 1];
            }
            Category->OthersCount--;
            break;
```

There were other functions that were used aside from the four major functions. There was the search function that searches for the product in a given category and returns the index where the product was located.

```
int SearchProduct(Product_Details *category, int count, char *Product_Name) {
  for (int i = 0; i < count; i++) {
    if (strcmp(category[i].product_name, Product_Name) == 0) {
      return i;
    }
  }
  return -1;
}
```

All of the functions utilize the same parameters and the same file to ensure uniformity in the code.

Transactions and Accounting Cycle

The structure used for the journal entries accounting the transactions on the store is the JournalEntry which contains data for the date, the credit, the debit and the note which elaborates on the economic activities occurring within the store.

```
struct JournalEntry{
    int year;
    int month;
    int day;
    char debit[50];
    char credit[50];
    double debit_amount;
    double credit_amount;
    char small_note[50];
};
```

The following functions in the main interface utilize the transaction:

```
case 5:
    InputSales(ItemP);
    break;
case 6: {
```

```
case 6: {
    char debit[50], credit[50], note[50];
    double amount;
    fflush(stdin);
    printf("Enter the debit account title: ");
    scanf( " %[^\n]s", debit);
    printf("Enter the credit account title: ");
    scanf( " %[^\n]s", credit);
    printf("Enter the note: ");
    scanf( " %[^\n]s", note);
    printf("Enter the amount: ");
    scanf("%lf", &amount);
    RecordTransaction(notebook, &transaction_count, debit,
credit, amount,
                      note);
```

```
case 7:
    PrintAllTransactions(notebook, transaction_count);
    break;
```

The Input Sales function inputs the quantity sold by users and follows the same interface as the Addition function, the Remover function and the Modifier function.

```c
void InputSales(Categories *Category) {
  char product_name[PROD_NAME];
  int category_choice, index;
  int quantity_sold;

  printf("\nSelect a Category for the Product Sale\n");
  printf("1. Beverages\n");
  printf("2. Canned Goods\n");
  printf("3. Dairy\n");
  printf("4. Dry Baking Goods\n");
  printf("5. Produce\n");
  printf("6. Cleaners\n");
  printf("7. Paper Goods\n");
  printf("8. Personal Care\n");
  printf("9. Others\n");
```

```c
printf("Option: ");
scanf("%d", &category_choice);

printf("Enter Product Name: ");
scanf(" %[^\n]s", product_name);
```

```c
  switch (category_choice) {
  case 1:
    index = SearchProduct(Category->Beverage, Category->BeverageCount,
                          product_name);
    break;
  case 2:
    index = SearchProduct(Category->Canned_Goods, Category->Canned_GoodsCount,
                          product_name);
    break;
```

The program asks for input from the user and similar to the Modify function, it uses switch cases and loops to check for the category where the product could be located. After the product is located, the program asks for the product's sold quantity.

```c
  }

  if (index == -1) {
    printf("Product not found.\n");
  } else {
    printf("Enter Quantity Sold: ");
    scanf("%d", &quantity_sold);
```

The quantity sold will then deduct the quantity available from the product.

```
  switch (category_choice) {
  case 1:
    Category->Beverage[index].quantity[0] -= quantity_sold;
    break;
  case 2:
    Category->Canned_Goods[index].quantity[0] -=
antity_sold;
    break;
  case 3:
    Category->Dairy[index].quantity[0] -= quantity_sold;
    break;
  case 4:
    Category->Dry_Baking_Goods[index].quantity[0] -=
antity_sold;
    break;
  case 5:
    Category->Produce[index].quantity[0] -= quantity_sold;
    break;
```

```
      Category->Produce[index].quantity[0] -= quantity_sold;
      break;
    case 6:
      Category->Cleaners[index].quantity[0] -= quantity_sold;
      break;
    case 7:
      Category->Paper_Goods[index].quantity[0] -=
quantity_sold;
      break;
    case 8:
      Category->Personal_Care[index].quantity[0] -=
quantity_sold;
      break;
    case 9:
      Category->Other[index].quantity[0] -= quantity_sold;
      break;
    }
    printf("Sales input successfully.\n");
  }
```

The accounting cycle program utilizes the following structures to store values relating to transaction:

Use of Structure and Malloc

The code for the journal and cost recording will mainly be derived from the concepts of **struct**, wherein key variables that help describe the product and its current inventory would be used to separate one product from another.

```
struct JournalEntry {
  int year;
  int month;
  int day;
  char debit[50];
  char credit[50];
  double debit_amount;
  double credit_amount;
  char small_note[50];
};
typedef struct JournalEntry JournalEntry;

JournalEntry notebook[MAX_TRANSACTIONS];
int transaction_count = 0;
```

Note Explainer

```
void NoteExplainer(char *buffer, int quantity, char *product_name, char *account_title){
    if(strcmp(account_title, "Sales")==0){
        sprintf(buffer, "Sold %d of %s", quantity, product_name);
    }else if(strcmp(account_title, "Purchases")==0){
        sprintf(buffer, "Bought %d of %s", quantity, product_name);
    }
} //to unify the selling and purchasing of inventory notes
```

This function helps the program generate short descriptions per modification of the inventory system. Like most transactions in a general journal, this program describes each transaction with the product name and the quantity bought/sold.

Record Transaction

```c
void RecordTransaction(int iterator, int transaction_count, JournalEntry* notebook, char account_title_1[], char account_title_2[], double amount, char small_note[]) {
    time_t currentTime;
    time(&currentTime);
    struct tm *localTime = localtime(&currentTime);

    int index = transaction_count + iterator - 1; // Adjust index calculation

    notebook[index].year = localTime->tm_year + 1900;
    notebook[index].month = localTime->tm_mon + 1;
    notebook[index].day = localTime->tm_mday;
    notebook[index].debit_amount = amount;
    notebook[index].credit_amount = amount;
    strcpy(notebook[index].debit, account_title_1);
    strcpy(notebook[index].credit, account_title_2);
    strcpy(notebook[index].small_note, small_note);

    printf("\n");

    //printf("%d/%d/%-8d%-40s%.2lf\n", notebook[index].month, notebook[index].day, notebook[index].year, notebook[index].debit, notebook[index].debit_amount);
    //printf("%14s%-50s%.2lf\n", "", notebook[index].credit, notebook[index].credit_amount);
    //printf("%-15s'%s'\n", "", notebook[index].small_note);
}
```

This function allows for the recording of transactions that take place within the business with external entities. In recording purchases and sales on account, adjusting necessary entries to reflect real-time transactions, and the like, the transactions serve as basis when used to reference transactions over time. This helps keep track of transactions that take place.

Printing of the General Journal

```c
void PrintAllTransactions(JournalEntry* notebook, int number_of_entries) {
    printf("%-20s%-33s%-12s%-10s\n", "Date", "Transaction", "Debit", "Credit");

    for (int counter = 0; counter < number_of_entries; counter++) { // Adjust loop condition
        printf("\n");
        printf("%d/%d/%-8d%-40s%.2lf\n", notebook[counter].month, notebook[counter].day, notebook[counter].year, notebook[counter].debit, notebook[counter].debit_amount);
        printf("%14s%-50s%.2lf\n", "", notebook[counter].credit, notebook[counter].credit_amount);
        printf("%-15s'%s'\n", "", notebook[counter].small_note);
    }

    printf("\n");
}
```

This function mainly prints all of the transactions that took place within the business's duration of operation. From start to finish, all transactions that were recorded in the program will be printed with this function.

General Ledger Function

```
void LedgerGeneratorNormalDebit(JournalEntry* notebook, char AccountTitle[], int number_of_entries){

    printf("%*s\n", 50, "General Ledger");
    printf("%*s\n", 45, AccountTitle);

    for(int counter = 0; counter < number_of_entries; counter++){
        if(strcmp(notebook[counter].debit,AccountTitle)==0){
            printf("%d/%d/%-8d%-40s%.2lf\n", notebook[counter].month, notebook[counter].day, notebook[counter].year, notebook[counter].small_note, notebook[counter].debit_amount);
            continue;
        }

        if(strcmp(notebook[counter].credit,AccountTitle)==0){
      printf("%d/%d/%-8d%-52s%.2lf\n", notebook[counter].month, notebook[counter].day, notebook[counter].year, notebook[counter].small_note, notebook[counter].credit_amount);
            continue;
        }
    }
    printf("\n");
}

void LedgerGeneratorNormalCredit(JournalEntry* notebook, char AccountTitle[], int number_of_entries){

    printf("%*s\n", 50, "General Ledger");
    printf("%*s\n", 45,AccountTitle);

    for(int counter = 0; counter < number_of_entries; counter++){
        if(strcmp(notebook[counter].credit,AccountTitle)==0){
        printf("%d/%d/%-8d%-53s%.2lf\n", notebook[counter].month, notebook[counter].day, notebook[counter].year, notebook[counter].small_note, notebook[counter].credit_amount);
        }

        if(strcmp(notebook[counter].debit, AccountTitle)==0){
      printf("%d/%d/%-8d%-40s%.2lf\n", notebook[counter].month, notebook[counter].day, notebook[counter].year, notebook[counter].small_note, notebook[counter].debit_amount);
        }
    }
    printf("\n");
}
```

The General Ledger is being treated as a function that prints only the specified account titles; the program filters the transactions to display the transactions with the relevant account titles. There are 2 functions to help describe the normal accounts per account.

Income Statement, SCE, SFP, SCF Generation Function

Should the sole proprietor want to find details as to how the profit/loss was earned, the program will generate a summary of all the variables, affected or not, in presenting numerical descriptions of the business during the current period.

The financial statements are divided into 4 functions:

> IncomeStatement

>StatementofChangesinEquity

>StatementofCashFlows

>BalanceSheet

The functions call the other functions in the program in order to perform the calculations needed to present the financial statements.

```
double IncomeStatement(JournalEntry* notebook, int number_of_entries, int month, int day, int year){
    printf("%*s\n", 50, "Aling Nena's Store");
    printf("%*s\n", 49, "Income Statement");
    printf("                          For the month ended %s %d, %d\n", getMonthName(month), day, year);

    double sales_amount = 0;
    for(int i=0; i < number_of_entries; i++){
        if(strcmp(notebook[i].credit,"Sales")==0){
            sales_amount = sales_amount + notebook[i].credit_amount;
        }else if(strcmp(notebook[i].debit,"Sales")==0){
            sales_amount = sales_amount - notebook[i].debit_amount;
        }
    }
    printf("%16s %54s %.2lf", "Gross Sales", "₱", sales_amount);
    printf("\n");

    printf("%9s", "Less");
    printf("\n");
    double salesdiscountsamount;
    for(int i=0; i < number_of_entries; i++){
        if(strcmp(notebook[i].debit,"Sales Discount")==0){
            salesdiscountsamount = salesdiscountsamount + notebook[i].debit_amount;
        }else if(strcmp(notebook[i].debit,"Sales Discount")==0){
            salesdiscountsamount = salesdiscountsamount - notebook[i].credit_amount;
        }
    }
    printf("      %s %35s %.2lf", "Sales Discounts", "₱", salesdiscountsamount);
    printf("\n");

    double sales_ra_amount = 0;
    for(int i=0; i < number_of_entries; i++){
        if(strcmp(notebook[i].debit,"Sales Returns and Allowances")==0){
            sales_ra_amount = sales_ra_amount + notebook[i].debit_amount;
        }else if(strcmp(notebook[i].debit,"Sales Returns and Allowances")==0){
            sales_ra_amount = sales_ra_amount - notebook[i].credit_amount;
        }
    }
    double less_sales_expenses;
    less_sales_expenses = salesdiscountsamount + sales_ra_amount;
    printf("      %s %25.2lf %17.2lf", "Sales Returns and Allowances", sales_ra_amount, less_sales_expenses);
    printf("\n");
    printf("                                                  ------------------------\n");
    double netsales_amount;
    netsales_amount = sales_amount - less_sales_expenses;
    printf("%14s %56s %.2lf", "Net Sales", "₱", netsales_amount);
    printf("\n");
    printf("\n");

    printf("%29s", "Less Cost of Goods Sold:");
    printf("\n");
    double beginning_inventory = 0;
    for(int i=0; i < number_of_entries; i++){
        if(strcmp(notebook[i].debit,"Merchandise Inventory, Beginning")==0){
            beginning_inventory = beginning_inventory + notebook[i].debit_amount;
        }
    }
    printf("      %s %16s %6.2lf", "Merchandise Inventory, Beginning", "₱", beginning_inventory);
    printf("\n");
    double total_purchases = 0;
    for(int i=0; i < number_of_entries; i++){
        if(strcmp(notebook[i].debit,"Purchases")==0){
            total_purchases = total_purchases + notebook[i].debit_amount;
        }if(strcmp(notebook[i].credit,"Purchases")==0){
            total_purchases = total_purchases - notebook[i].credit_amount;
        }
    }
    printf("%21s %38.2lf", "Gross Purchases", total_purchases);
    printf("\n");
    double transpo_in = 0;
    for(int i=0; i < number_of_entries; i++){
```

```
double purchase_discounts = 0;
for(int i=0; i < number_of_entries; i++){
    if(strcmp(notebook[i].credit,"Purchase Discounts")==0){
        purchase_discounts = purchase_discounts + notebook[i].credit_amount;
    }
}

printf("      %s %19s %6.2lf", "Purchase Discounts", "₱", purchase_discounts);
printf("\n");
double purchase_ra = 0;
for(int i=0; i < number_of_entries; i++){
    if(strcmp(notebook[i].credit,"Purchase Returns and Allowances")==0){
        purchase_discounts = purchase_discounts + notebook[i].credit_amount;
    }
}
double total_purchase_expenses;
total_purchase_expenses = purchase_discounts + purchase_ra;
printf("      %s %11.2lf %10.2lf", "Purchase Returns and Allowances", purchase_discounts, total_purchase_expenses);
printf("\n");
printf("                                          ----------------------\n");
double tgas;
tgas = beginning_inventory + total_purchases + transpo_in - total_purchase_expenses;
printf("      %s%16s%10.2lf", "Total Goods Available for Sale", "₱", tgas);
printf("\n");
//iterate all the quantity remaining in the inventory
double ending_inventory;
ending_inventory = 0; //insert all the remaining quantity in the inventory
double total_cogs;
total_cogs = tgas - ending_inventory;
printf("      %14s %19.2lf %17.2lf", "Less: Merhandise Inventory, Ending", ending_inventory, total_cogs);
printf("\n");
printf("                                            ------------------------------------\n");
double gross_income;
gross_income = netsales_amount - total_cogs;
printf("%18s %52s %2.2lf", "Gross Income", "₱", gross_income);
printf("\n");
printf("\n");
printf("%18s %52s %2.2lf", "Total Income", "₱",gross_income);
printf("\n");
printf("\n");
printf("%19s:", "Less Expenses");
printf("\n");
double utility_expense;
double supplies_expense;
double miscellaneous_expense;
double total_expenses;
for(int i=0; i < number_of_entries; i++){
    if(strcmp(notebook[i].debit,"Utility Expense")==0){
        utility_expense = utility_expense + notebook[i].debit_amount;
    }
}
for(int i=0; i < number_of_entries; i++){
    if(strcmp(notebook[i].debit,"Supplies Expense")==0){
        supplies_expense = supplies_expense + notebook[i].debit_amount;
    }
}
for(int i=0; i < number_of_entries; i++){
    if(strcmp(notebook[i].debit,"Miscellaneous Expense")==0){
        miscellaneous_expense = miscellaneous_expense + notebook[i].debit_amount;
    }
}
total_expenses = utility_expense + supplies_expense + miscellaneous_expense;
printf("%24s %35.2lf", "Utility Expense", utility_expense);
printf("\n");
printf("%25s %34.2lf", "Supplies Expense", supplies_expense);
printf("\n");
printf("%30s %29.2lf %17.2lf", "Miscellaneous Expense", supplies_expense, total_expenses);
printf("\n");
printf("                                                  ----------------------------\n");
```

```
    double profit_loss = 0;
    profit_loss = gross_income - total_expenses;
    //profit_loss = -10000.00;

    if(profit_loss > 0){
        printf("%16s                                                %s %.2lf", "Net Profit", "₱", profit_loss);
        printf("\n");
        printf("                                                              ==========");
    }else if(profit_loss < 0){
        profit_loss = profit_loss * -1;
        printf("%14s                                                 %s (%.2lf)", "Net Loss", "₱", profit_loss);
        printf("\n");
        printf("                                                              ==========");
        profit_loss = profit_loss * -1;
    }

    printf("\n");

    return profit_loss;
}

double StatementofChangesinEquity(JournalEntry* notebook, int number_of_entries, int month, int day, int last_day, int year, double profit_loss){

    printf("%*s\n", 50, "Aling Nena's Store");
    printf("%*s\n", 56, "Statement of Changes in Equity");
    printf("                        For the month ended %s %d, %d\n", getMonthName(month), day, year);

    double changes_in_equity=0;
    printf("\n");

    for(int i = 0; i < number_of_entries ; i++){
        if(strcmp(notebook[i].small_note, "Beginning Balance") == 0 && strcmp(notebook[i].credit, "Nena, Capital")==0){
            printf("%19s - %s %d, %d %34s %10.2lf", "Nena, Capital", getMonthName(notebook[i].month), notebook[i].day, notebook[i].year, "₱", notebook[i].credit_amount);
            printf("\n");
            changes_in_equity = notebook[i].credit_amount;
        }
    }

    if(profit_loss > 0){
        printf("%21s %56.2lf", "Add: Net Profit", profit_loss);
        printf("\n");
        printf("                                                              ------------");
        printf("\n");
    }else if(profit_loss <0){
        profit_l                                                  * -1;
        printf("                                                 (%.2lf)", "Less: Net Loss", profit_loss);
        printf("\n");
        printf("                                                              ------------");
        printf("\n");
        profit_loss = profit_loss * -1;
    }
    double total;
    total = changes_in_equity + profit_loss;
    printf("%11s %57s %10.2lf", "Total", "₱", total);
    printf("\n");

    double withdrawals;
    for(int i = 0; i < number_of_entries; i++){
        if(strcmp(notebook[i].debit, "Nena, Withdrawals")==0){-
    }
    printf("%29s %38s (%.2lf)", "Less: Nena, Withdrawals", "", withdrawals);
    printf("\n");

    double final_equity;
    final_equity = total - withdrawals;
    printf("                                                            ------------");
    printf("\n");
    printf("%19s - %s %d, %d %33s %10.2lf", "Nena, Capital", getMonthName(month), last_day, year, "₱", final_equity);
    printf("\n");
    printf("                                                            ============");
    printf("\n");

    return final_equity;
}
```

(char [2])"\n"

```c
double StatementOfCashFlows(JournalEntry *notebook, int number_of_entries, int month, int day, int year){

    printf("%*s\n", 50, "Aling Nena's Store");
    printf("%*s\n", 53, "Statement of Cash Flows");
    printf("                         For the month ended %s %d, %d\n", getMonthName(month), day, year);

printf("\n");
//flow from operating activities

    double operating_activities = 0;
    double investing_activities = 0;
    double financing_activities = 0;
    printf("%s\n", "Cash Flows from Operating Activities");
    for(int i=0; i <number_of_entries; i++){
        if(i==0){
            printf("%15s", "Receipts:");
            printf("\n");
        }
            if(strcmp(notebook[i].debit,"Cash")== 0 && strcmp(notebook[i].credit, "Sales")==0){
                printf("%20s %37s %10.2lf\n",notebook[i].small_note, "P", notebook[i].debit_amount);
            operating_activities = operating_activities + notebook[i].debit_amount;
            }else if(strcmp(notebook[i].debit,"Cash")== 0 && strcmp(notebook[i].credit, "Accounts Receivable")==0){
                printf("%20s\n",notebook[i].small_note);
            operating_activities = operating_activities + notebook[i].debit_amount;
            }
    }

        printf("\n");

    for(int i=0; i<number_of_entries; i++){

        if(i==0){
            printf("%20s", "Disbursements:");
            printf("\n");
        }
            if(strcmp(notebook[i].debit,"Purchases")== 0 && strcmp(notebook[i].credit, "Cash")==0){
                printf("%22s %35s(%.2lf)\n",notebook[i].small_note, "", notebook[i].credit_amount);
                operating_activities = operating_activities - notebook[i].credit_amount;
            }else if(strcmp(notebook[i].debit,"Utility Expense")== 0 && strcmp(notebook[i].credit, "Cash")==0){
                printf("%24s %32s (%.2lf)\n",notebook[i].small_note, "", notebook[i].credit_amount);
                operating_activities = operating_activities - notebook[i].credit_amount;
            }else if(strcmp(notebook[i].debit,"Supplies Expense")== 0 && strcmp(notebook[i].credit, "Cash")==0){
                printf("%24s%34s (%.2lf)\n",notebook[i].small_note, "", notebook[i].credit_amount);
                operating_activities = operating_activities - notebook[i].credit_amount;
            }else if(strcmp(notebook[i].debit,"Miscellaneous Expense")== 0 && strcmp(notebook[i].credit, "Cash")==0){
                printf("%24s%34s (%.2lf)\n",notebook[i].small_note, "", notebook[i].credit_amount);
                operating_activities = operating_activities - notebook[i].credit_amount;
            }else if(strcmp(notebook[i].debit,"Transportation In")== 0 && strcmp(notebook[i].credit, "Cash")==0){
                printf("%24s%34s (%.2lf)\n",notebook[i].small_note, "", notebook[i].credit_amount);
                operating_activities = operating_activities - notebook[i].credit_amount;
            }else if(strcmp(notebook[i].debit,"Accounts Payable")== 0 && strcmp(notebook[i].credit, "Cash")==0){
                printf("%24s%34s (%.2lf)\n",notebook[i].small_note, "", notebook[i].credit_amount);
            }
        }
    printf("%54s ------------\n", "");
    printf("\n");

    if(operating_activities < 0){
        operating_activities = operating_activities * -1;
        printf("%52s %18s %s (%.2lf)", "Net Cash Flow Provided by Operating Activities", "P", "", operating_activities);
        operating_activities = operating_activities * -1;
    }else if (operating_activities == 0){
        printf("%.2lf", operating_activities);
    }else if (operating_activities > 0){
        printf("%52s %18s %10.2lf", "Net Cash Flow Provided by Operating Activities", "P", operating_activities);
    }

    printf("\n");
    printf("\n");
```

```
printf("%s\n", "Cash Flows from Investing Activities");
    //no investing activities
printf("\n");
investing_activities = 0;
if(investing_activities < 0){
    investing_activities = investing_activities * -1;
    printf("%52s %18s %5s (%.2lf)", "Net Cash Flow Provided by Investing Activities", "₱", "", investing_activities);
    investing_activities = investing_activities * -1;
}else if (investing_activities == 0){
    printf("%52s %18s %7s", "Net Cash Flow Provided by Investing Activities", "₱", "-");
}else{
    printf("%52s %18s %3.2lf", "Net Cash Flow Provided by Investing Activities", "₱", investing_activities);
}
printf("\n");

printf("\n");
printf("%s\n", "Cash Flows from Financing Activities");
printf("%16s", "Receipts:\n");
    for(int i=0; i<number_of_entries; i++){
        if(strcmp(notebook[i].debit,"Cash")== 0 && strcmp(notebook[i].credit, "Nena, Capital") == 0 && strcmp(notebook[i].small_note,"Beginning Balance")>0){
            printf("%23s %44s %s %.2lf\n",notebook[i].small_note, "₱", "", notebook[i].debit_amount);
            financing_activities = financing_activities + notebook[i].debit_amount;
        }
    }
printf("\n");
printf("%20s", "Disbursements:");
printf("\n");
    for(int i=0; i<number_of_entries; i++){
        if(strcmp(notebook[i].debit,"Nena, Withdrawals")== 0 && strcmp(notebook[i].credit, "Cash")==0){
            printf("%37s %20s (%.2lf)\n",notebook[i].small_note, "", notebook[i].credit_amount);
            financing_activities = financing_activities - notebook[i].credit_amount;
        }
    }
printf("%55s ------------\n", "");
printf("\n");
if(financing_activities < 0){
    financing_activities = financing_activities * -1;
    printf("%52s %18s %s (%.2lf)", "Net Cash Flow Provided by Financing Activities", "₱", "", financing_activities);
    financing_activities = financing_activities * -1;
}else if (financing_activities == 0){
    printf("%52s %18s %5s %s", "Net Cash Flow Provided by Financing Activities", "₱", "", "-");
}else if (financing_activities > 0){
    printf("%52s %18s %s %2.2lf", "Net Cash Flow Provided by Financing Activities", "₱", "", financing_activities);
}
printf("\n");
printf("%67s ------------\n", "");
printf("\n");
double ending_cash = operating_activities + investing_activities + financing_activities;

if(ending_cash < 0){
    ending_cash = ending_cash * -1;
    printf("%19s %50s (%.2lf)", "Net Cash Flow", "", ending_cash);
    ending_cash = ending_cash * -1;
    printf("\n");
}else{
    printf("%19s %s %60.2lf", "Net Cash Flow", "", ending_cash);
    printf("\n");
}


for(int i=0; i<number_of_entries; i++){
    if(strcmp(notebook[i].small_note, "Beginning Balance")==0){
        if(notebook[i].debit_amount < 0){
            notebook[i].debit_amount = notebook[i].debit_amount * -1;
            printf("%20s %52s (%.2lf)", "Less: Cash, Beginning", "", notebook[i].debit_amount);
            printf("\n");
            notebook[i].debit_amount = notebook[i].debit_amount * -1;
            ending_cash = ending_cash + notebook[i].debit_amount;
        }else{
            printf("%26s %54.2lf", "Add: Cash, Beginning", notebook[i].debit_amount);
            printf("\n");
            ending_cash = ending_cash + notebook[i].debit_amount;
        }
    }
}
printf("%67s ------------\n", "");
```

```
    }
printf("%55s ------------\n", "");
printf("\n");
if(financing_activities < 0){
    financing_activities = financing_activities * -1;
    printf("%52s %18s %s (%.2lf)", "Net Cash Flow Provided by Financing Activities", "P", "", financing_activities);
    financing_activities = financing_activities * -1;
}else if (financing_activities == 0){
    printf("%52s %18s %5s %s", "Net Cash Flow Provided by Financing Activities", "P", "", "-");
}else if (financing_activities > 0){
    printf("%52s %18s %s %2.2lf", "Net Cash Flow Provided by Financing Activities", "P", "", financing_activities);
}
printf("\n");
printf("%67s ------------\n", "");
printf("\n");
double ending_cash = operating_activities + investing_activities + financing_activities;

if(ending_cash < 0){
    ending_cash = ending_cash * -1;
    printf("%19s %50s (%.2lf)", "Net Cash Flow", "", ending_cash);
    ending_cash = ending_cash * -1;
    printf("\n");
}else{
    printf("%19s %s %60.2lf", "Net Cash Flow", "", ending_cash);
    printf("\n");
}


for(int i=0; i<number_of_entries; i++){
    if(strcmp(notebook[i].small_note, "Beginning Balance")==0){
        if(notebook[i].debit_amount < 0){
            notebook[i].debit_amount = notebook[i].debit_amount * -1;
            printf("%20s %52s (%.2lf)", "Less: Cash, Beginning", "", notebook[i].debit_amount);
            printf("\n");
            notebook[i].debit_amount = notebook[i].debit_amount * -1;
            ending_cash = ending_cash + notebook[i].debit_amount;
        }else{
            printf("%26s %54.2lf", "Add: Cash, Beginning", notebook[i].debit_amount);
            printf("\n");
            ending_cash = ending_cash + notebook[i].debit_amount;
        }
    }
}
printf("%67s ------------\n", "");

if(ending_cash <0){
    ending_cash = ending_cash * -1;
    printf("%10s - %s %d, %d %52s (%.2lf)", "Cash", getMonthName(month), day, year, "", ending_cash);
    printf("\n");
    ending_cash = ending_cash * -1;
}else{
    printf("%10s - %s %d, %d %45s %11.2lf", "Cash", getMonthName(month), day, year, "P", ending_cash);
    printf("\n");
}
printf("%67s ============\n", "");


return ending_cash;
```

```c
void BalanceSheet(JournalEntry *notebook, int number_of_entries, int month, int day, int year, double new_equity){
    printf("%*s\n", 50, "Aling Nena's Store");
    printf("%*s\n", 57, "Statement of Financial Position");
    printf("%33s %s %d, %d\n", "", getMonthName(month), day, year);
    printf("\n");

    double cash = BalanceCalculatorNormalDebit(notebook, "Cash", number_of_entries);
    double acc_rec = BalanceCalculatorNormalDebit(notebook, "Accounts Receivable", number_of_entries);
    double merch_inv = BalanceCalculatorNormalDebit(notebook, "Merchandise Inventory", number_of_entries);
    double supplies = BalanceCalculatorNormalDebit(notebook, "Supplies", number_of_entries);
    double current_assets = cash + acc_rec + merch_inv + supplies;

    printf("%44s", "ASSETS");
    printf("\n");

    printf("%21s","Current Assets:");
    printf("\n");
    printf("%15s %42s %10.2lf","Cash", "₱", cash);
    printf("\n");

    printf("%30s %36.2lf","Accounts Receivable", acc_rec);
    printf("\n");

    printf("%32s %34.2lf","Merchandise Inventory", merch_inv);
    printf("\n");

    printf("%19s %47.2lf","Supplies", supplies);
    printf("\n");
    printf("%53s ----------\n", "");
    printf("\n");

    printf("%31s %40s %10.2lf","Total Current Assets", "₱", current_assets);
    printf("\n");

    printf("\n");

    printf("%24s","Noncurrent Assets:");
    printf("\n");
    double land = BalanceCalculatorNormalDebit(notebook, "Land", number_of_entries);
    double building = BalanceCalculatorNormalDebit(notebook, "Building", number_of_entries);
    double accum_dep_building = BalanceCalculatorNormalCredit(notebook, "Accumulated Depreciation - Building", number_of_entries);
    double equipment = BalanceCalculatorNormalDebit(notebook, "Equipment", number_of_entries);
    double accum_dep_equipment = BalanceCalculatorNormalCredit(notebook, "Accumulated Depreciation - Equipment", number_of_entries);
    double noncurrent_assets = land + building - accum_dep_building + equipment - accum_dep_equipment;

    printf("%15s %42s %10.2lf","Land", "₱", land);
    printf("\n");

    printf("%19s %47.2lf","Building", building);
    printf("\n");

    printf("%46s %15s(%.2lf)","Accumulated Depreciation - Building", "", accum_dep_building);
    printf("\n");

    printf("%20s %46.2lf","Equipment", equipment);
    printf("\n");

    printf("%47s %14s(%.2lf)","Accumulated Depreciation - Equipment", "", accum_dep_equipment);
    printf("\n");
    printf("%54s ----------\n", "");

    printf("\n");

    printf("%34s %37s %11.2lf","Total Noncurrent Assets", "₱", noncurrent_assets);
    printf("\n");
    printf("%68s ----------\n", "");

    printf("\n");

    double total_assets = current_assets + noncurrent_assets;
    printf("%18s %53s %11.2lf","TOTAL ASSETS", "₱", total_assets);
    printf("\n");
    printf("%68s ==========\n", "");

    printf("\n");

    printf("%55s", "LIABILITIES AND OWNER'S EQUITY");
    printf("\n");
```

```
    printf("%55s", "LIABILITIES AND OWNER'S EQUITY");
    printf("\n");

    printf("\n");

    printf("%26s","Current Liabilities:");
    printf("\n");
    double accounts_payable = BalanceCalculatorNormalCredit(notebook, "Accounts Payable", number_of_entries);
    double capital = new_equity;

    printf("%27s %31s %11.2lf","Accounts Payable", "₱", accounts_payable);
    printf("\n");
    printf("%55s ------------\n", "");

    double current_liabilities = accounts_payable;

    printf("%36s %35s %11.2lf","Total Current Liabilities", "₱", current_liabilities);
    printf("\n");

    printf("\n");

    printf("%20s %51s %11.2lf","Nena, Capital", "₱", new_equity);
    printf("\n");
    printf("%68s ------------\n", "");

    printf("\n");

    double total_liabeq = current_liabilities + new_equity;

    printf("%42s %29s %11.2lf","TOTAL LIABILITIES AND OWNER'S EQUITY", "₱", total_liabeq);
    printf("\n");
    printf("%68s ============\n", "");
}
```

Balance Calculator

```
double BalanceCalculatorNormalDebit(JournalEntry *notebook, char AccountTitle[], int number_of_entries){
    double total_sum = 0;
    for(int i=0; i<number_of_entries; i++){
        if(strcmp(notebook[i].debit, AccountTitle)==0){
            total_sum = total_sum + notebook[i].debit_amount;
            continue;
        }else if(strcmp(notebook[i].credit, AccountTitle)==0){
            total_sum = total_sum - notebook[i].credit_amount;
            continue;
        }
    }
    return total_sum;
}

double BalanceCalculatorNormalCredit(JournalEntry *notebook, char AccountTitle[], int number_of_entries){
    double total_sum = 0;
    for(int i=0; i<number_of_entries; i++){
        if(strcmp(notebook[i].credit, AccountTitle)==0){
            total_sum = total_sum + notebook[i].credit_amount;
        }else if(strcmp(notebook[i].debit, AccountTitle)==0){
            total_sum = total_sum - notebook[i].debit_amount;
        }
    }
    return total_sum;
}
```

Unlike the ledger, this function allows the total calculation of an account title's ending balances.

Limitations

- The current recording transaction function does not allow for multiple account titles in one transaction. Hence, the user is left inputting the respective accounts one by one.

Date		Transaction	P.R.	Debit	Credit
FEB	28	Sales	410	1,332,000	
		Income Summary	330		1,179,160
		Sales Returns and Allowances	420		141,000
		Sales Discounts	430		11,840
		To close nominal income accounts			

For example, most closing transactions look like this, but the user is forced to input their transactions like this instead:

```
6/3/2024    Sales                                  25000.00
              Income Summary                                    25000.00
               'To close nominal income accounts'

6/3/2024    Income Summary                         0.00
              Sales Returns and Allowances                      0.00
               'To close nominal income accounts'

6/3/2024    Income Summary                         0.00
              Sales Discounts                                   0.00
               'To close nominal income accounts'

6/3/2024    Purchase Returns and Allowances        0.00
```

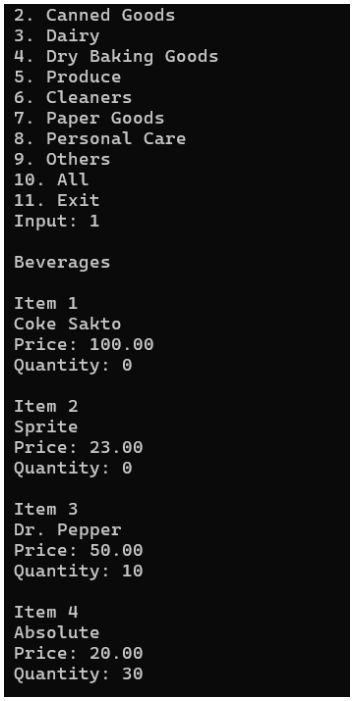
Although unproductive, the recording function still does its job at keeping the account titles balanced.

- The program is limited to pre-existing account titles, especially for checking the titles in iterations. New account titles have yet to be recognized, unless the developers opt to develop an enhanced version of the recording function.
- The program has inconsistent formatting, especially with long and short texts. Print-based outputs are limited on a “width” measurement to ensure equal distances across all formats, especially in the financial statements. However, unexpected but valid inputs may not cause the program to print the way it was programmed to print. Centering of variables and monetary descriptions may be off in some cases.

Code Snippets:

```
Beverage|Coke Sakto|100.00|grams|20.00|pcs|0|15
Canned_Goods|Century Tuna Spicy|200.00|grams|30.00|pcs|0|25.00
Dairy|Eden Cheese|50.00|grams|15.00|pcs|0|13.00
Dry_Baking_Goods|All-purpose Flour|100.00|grams|40.00|pcs|0|30.00
Produce|Red Onion|0.25|grams|25.00|pcs|0|0.10
Cleaners|Tide|10.00|grams|8.00|pcs|0|5.00
Paper_Goods|Toilet Paper|50.00|grams|8.00|pcs|0|4.00
Personal_Care|Safe Guard Soap|10.00|grams|8.00|pcs|0|5.00
Other|Short Bondpaper|5.00|grams|1.00|pcs|0|0.50
Beverage|Sprite|23.00|grams|12.33|pcs|0|10.00
Canned_Goods|555 Sardines|55.00|grams|55.00|pcs|0|40.00
Cleaners|Surf Powder Blue|6.00|grams|10.00|pcs|0|4.00
Beverage|Dr. Pepper|50.00|gramws|50.00|pcs|10|400.00
Beverage|Absolute|20.00|grams|20.00|pcs|30|15.000000
```

Initial Product.txt with initial products of the store



Sample Implementation of Display Product function

```
8. Exit
Input: 2

Select a Category for the New Product
1. Beverages
2. Canned Goods
3. Dairy
4. Dry Baking Goods
5. Produce
6. Cleaners
7. Paper Goods
8. Personal Care
9. Others
Option: 1
Enter New Product Name: Mountain Dew
Enter Product Price: 27.00
Enter Product Weight: 50
Enter Weight Category: grams
Enter Product Quantity: 10
Enter Quantity Category: pcs
Enter Cost: 24.00

Transaction recorded successfully:
6/3/2024
Debit: Purchases, Credit: Cash, Amount: 240.00, Note: Bought 10 of Mountain Dew
New product added successfully.
```

Sample Implementation of Add Function

```
Beverage|Coke Sakto|100.00|grams|20.00|pcs|0|15
Canned_Goods|Century Tuna Spicy|200.00|grams|30.00|pcs|0|25.00
Dairy|Eden Cheese|50.00|grams|15.00|pcs|0|13.00
Dry_Baking_Goods|All-purpose Flour|100.00|grams|40.00|pcs|0|30.0
Produce|Red Onion|0.25|grams|25.00|pcs|0|0.10
Cleaners|Tide|10.00|grams|8.00|pcs|0|5.00
Paper_Goods|Toilet Paper|50.00|grams|8.00|pcs|0|4.00
Personal_Care|Safe Guard Soap|10.00|grams|8.00|pcs|0|5.00
Other|Short Bondpaper|5.00|grams|1.00|pcs|0|0.50
Beverage|Sprite|23.00|grams|12.33|pcs|0|10.00
Canned_Goods|555 Sardines|55.00|grams|55.00|pcs|0|40.00
Cleaners|Surf Powder Blue|6.00|grams|10.00|pcs|0|4.00
Beverage|Dr. Pepper|50.00|gramws|50.00|pcs|10|400.00
Beverage|Absolute|20.00|grams|20.00|pcs|30|15.00
Beverage|Mountain Dew|27.00|grams|50.00|pcs|10|24.00
```

Updated Product.txt

```
1. Display Products
2. Add Products
3. Modify Product Detail
4. Remove a Product
5. Input Sales
6. Record a Transaction
7. Print All Transactions
8. Exit
Input: 3

Select a Category to Modify the Product
1. Beverages
2. Canned Goods
3. Dairy
4. Dry Baking Goods
5. Produce
6. Cleaners
7. Paper Goods
8. Personal Care
9. Others
Option: 1
Enter the Product Name to Modify: Mountain Dew
Select the Detail to Modify:
1. Product Name
2. Price
3. Product Weight
4. Weight Category
5. Quantity
6. Quantity Category
Option: 1
Enter New Product Name: Fanta
Product modified successfully.
```

Sample Implementation of Modifier Function

```
11. Exit
Input: 1

Beverages

Item 1
Coke Sakto
Price: 100.00
Quantity: 0

Item 2
Sprite
Price: 23.00
Quantity: 0

Item 3
Dr. Pepper
Price: 50.00
Quantity: 10

Item 4
Absolute
Price: 20.00
Quantity: 30

Item 5
Fanta
Price: 27.00
Quantity: 10
```

Item 5’s product name is modified

```
Beverage|Coke Sakto|100.00|grams|20.00|pcs|0|15
Canned_Goods|Century Tuna Spicy|200.00|grams|30.00|pcs|0|25.00
Dairy|Eden Cheese|50.00|grams|15.00|pcs|0|13.00
Dry_Baking_Goods|All-purpose Flour|100.00|grams|40.00|pcs|0|30.00
Produce|Red Onion|0.25|grams|25.00|pcs|0|0.10
Cleaners|Tide|10.00|grams|8.00|pcs|0|5.00
Paper_Goods|Toilet Paper|50.00|grams|8.00|pcs|0|4.00
Personal_Care|Safe Guard Soap|10.00|grams|8.00|pcs|0|5.00
Other|Short Bondpaper|5.00|grams|1.00|pcs|0|0.50
Beverage|Sprite|23.00|grams|12.33|pcs|0|10.00
Canned_Goods|555 Sardines|55.00|grams|55.00|pcs|0|40.00
Cleaners|Surf Powder Blue|6.00|grams|10.00|pcs|0|4.00
Beverage|Dr. Pepper|50.00|gramws|50.00|pcs|10|400.00
Beverage|Absolute|20.00|grams|20.00|pcs|30|15.000000
Beverage|Fanta|27.00|grams|50.00|pcs|10|24.00
```

Updated Product.txt

```
                        WELCOME USER!

1. Display Products
2. Add Products
3. Modify Product Detail
4. Remove a Product
5. Input Sales
6. Record a Transaction
7. Print All Transactions
8. Exit
Input: 4

Select a Category to Remove Product
1. Beverages
2. Canned Goods
3. Dairy
4. Dry Baking Goods
5. Produce
6. Cleaners
7. Paper Goods
8. Personal Care
9. Others
Option: 1
Enter the Product Name to Remove: Coke Sakto
Product "Coke Sakto" removed successfully
```

Sample Implementation of Remove Function

```
Beverages

Item 1
Sprite
Price: 23.00
Quantity: 0

Item 2
Dr. Pepper
Price: 50.00
Quantity: 10

Item 3
Absolute
Price: 20.00
Quantity: 30

Item 4
Fanta
Price: 27.00
Quantity: 10
```

Beverage Catalog after removal of Coke Sakto

```
Canned_Goods|Century Tuna Spicy|200.00|grams|30.00|pcs|0|25.00
Dairy|Eden Cheese|50.00|grams|15.00|pcs|0|13.00
Dry_Baking_Goods|All-purpose Flour|100.00|grams|40.00|pcs|0|30.00
Produce|Red Onion|0.25|grams|25.00|pcs|0|0.10
Cleaners|Tide|10.00|grams|8.00|pcs|0|5.00
Paper_Goods|Toilet Paper|50.00|grams|8.00|pcs|0|4.00
Personal_Care|Safe Guard Soap|10.00|grams|8.00|pcs|0|5.00
Other|Short Bondpaper|5.00|grams|1.00|pcs|0|0.50
Beverage|Sprite|23.00|grams|12.33|pcs|0|10.00
Canned_Goods|555 Sardines|55.00|grams|55.00|pcs|0|40.00
Cleaners|Surf Powder Blue|6.00|grams|10.00|pcs|0|4.00
Beverage|Dr. Pepper|50.00|gramws|50.00|pcs|10|400.00
Beverage|Absolute|20.00|grams|20.00|pcs|30|15.000000
Beverage|Fanta|27.00|grams|50.00|pcs|10|24.00
```

Updated Product.txt

```
1. Display Products
2. Add Products
3. Modify Product Detail
4. Remove a Product
5. Input Sales
6. Record a Transaction
7. Print All Transactions
8. Exit
Input: 6
Enter the debit account title: Cash
Enter the credit account title: Sales
Enter the note: Sold 10 Eco-Friendly Wood Veneers
Enter the amount: 10000.00

Transaction recorded successfully:
6/3/2024
Debit: Cash, Credit: Sales, Amount: 10000.00, Note: Sold 10 Eco-Friendly Wood Veneers

1. Display Products
2. Add Products
3. Modify Product Detail
4. Remove a Product
5. Input Sales
6. Record a Transaction
7. Print All Transactions
8. Exit
Input: 7
Date              Transaction                    Debit       Credit
6/3/2024    Cash                                 10000.00
              Sales                                           10000.00
               'Sold 10 Eco-Friendly Wood Veneers'
```

Sample Implementation of Transaction Function

```
               'Sold 10 of Galvanized Steel'
Date                Transaction                    Debit       Credit
5/3/2024    Cash                                 10000.00
              Sales                                           10000.00
               'Sold 10 Eco-Friendly Wood Veneers'
```

Transaction saved in transaction.txt

General Ledger
Cash

6/3/2024	Beginning Balance	40000.00	
6/3/2024	Bought stuff		20000.00
6/3/2024	Sold stuff	5000.00	
6/3/2024	Withdrew Cash for fun		10000.00
6/4/2024	John made utang		10000.00
6/5/2024	Nena made utang		10000.00

General Ledger
Purchases

6/3/2024	Bought stuff	20000.00	
6/5/2024	Nena made utang	10000.00	

General Ledger
Sales

6/3/2024	Sold stuff		5000.00

General Ledger
Nena, Capital

6/3/2024	Beginning Balance		40000.00

Aling Nena's Store
Income Statement
For the month ended June 3, 2024

Gross Sales			₱ 5000.00
Less			
Sales Discounts		₱ 40000.00	
Sales Returns and Allowances		0.00	40000.00
Net Sales			₱ -35000.00
Less Cost of Goods Sold:			
Merchandise Inventory, Beginning		₱ 0.00	
Gross Purchases		30000.00	
Transportation In		0.00	
Less:			
Purchase Discounts	₱ 0.00		
Purchase Returns and Allowances	0.00	0.00	
Total Goods Available for Sale		₱ 30000.00	
Less: Merhandise Inventory, Ending		0.00	30000.00
Gross Income			₱ -65000.00
Total Income			₱ -65000.00

```
      Less Expenses:
         Utility Expense                          0.00
         Supplies Expense                         0.00
         Miscellaneous Expense                    0.00              0.00
                                                                  ----------
      Net Loss                                                   ₱ (65000.00)
                                                                  ==========

                              Aling Nena's Store
                         Statement of Changes in Equity
                        For the month ended June 1, 2024

      Nena, Capital - June 3, 2024                               ₱   40000.00
      Less: Net Loss                                               (65000.00)
                                                                  ----------
      Total                                                      ₱  -25000.00
      Less: Nena, Withdrawals                                      (10000.00)
                                                                  ----------
      Nena, Capital - June 31, 2024                              ₱  -35000.00
                                                                  ==========

                              Aling Nena's Store
                           Statement of Cash Flows
                        For the month ended June 1, 2024

Cash Flows from Operating Activities
      Receipts:
          Sold stuff                                 ₱    5000.00

      Disbursements:
          Bought stuff                                  (20000.00)
       Nena made utang                                  (10000.00)
                                                      ------------

      Net Cash Flow Provided by Operating Activities              ₱  (25000.00)

Cash Flows from Investing Activities

      Net Cash Flow Provided by Investing Activities              ₱       -

Cash Flows from Financing Activities
      Receipts:

      Disbursements:
                Withdrew Cash for fun                  (10000.00)
                                                     ------------

      Net Cash Flow Provided by Financing Activities              ₱  (10000.00)
                                                                  ------------
```

Cash Flows from Financing Activities

Receipts:		
Disbursements:		
Withdrew Cash for fun	(10000.00)	
Net Cash Flow Provided by Financing Activities		₱ (10000.00)
Net Cash Flow		(35000.00)
Add: Cash, Beginning		40000.00
Cash - June 1, 2024		₱ 5000.00

Aling Nena's Store
Statement of Financial Position
June 31, 2024

ASSETS

Current Assets:		
Cash	₱ -5000.00	
Accounts Receivable	10000.00	
Merchandise Inventory	0.00	
Supplies	0.00	
Total Current Assets		₱ 5000.00
Noncurrent Assets:		
Land	₱ 0.00	
Building	0.00	
Accumulated Depreciation - Building	(0.00)	
Equipment	0.00	
Accumulated Depreciation - Equipment	(0.00)	
Total Noncurrent Assets		₱ 0.00
TOTAL ASSETS		₱ 5000.00

LIABILITIES AND OWNER'S EQUITY

Current Liabilities:		
Accounts Payable	₱ 0.00	
Total Current Liabilities		₱ 0.00
Nena, Capital		₱ -35000.00
TOTAL LIABILITIES AND OWNER'S EQUITY		₱ -35000.00

Date	Transaction	Debit	Credit
6/3/2024	Cash	40000.00	
	Nena, Capital		40000.00
	'Beginning Balance'		
6/3/2024	Purchases	20000.00	
	Cash		20000.00
	'Bought stuff'		
6/3/2024	Cash	5000.00	
	Sales		5000.00
	'Sold stuff'		
6/3/2024	Nena, Withdrawals	10000.00	
	Cash		10000.00
	'Withdrew Cash for fun'		
6/4/2024	Accounts Receivable	10000.00	
	Cash		10000.00
	'John made utang'		
6/5/2024	Purchases	10000.00	
	Cash		10000.00
	'Nena made utang'		
6/3/2024	Sales	5000.00	
	Income Summary		5000.00
	'To close nominal income accounts'		
6/3/2024	Income Summary	0.00	
	Sales Returns and Allowances		0.00
	'To close nominal income accounts'		
6/3/2024	Income Summary	0.00	
	Sales Discounts		0.00
	'To close nominal income accounts'		
6/3/2024	Purchase Returns and Allowances	0.00	
	Income Summary		0.00
	'To close nominal expense accounts'		
6/3/2024	Purchase Discounts	0.00	
	Income Summary		0.00
	'To close nominal expense accounts'		
6/3/2024	Income Summary	30000.00	
	Purchases		30000.00
	'To close nominal expense accounts'		
6/3/2024	Income Summary	0.00	
	Transportation In		0.00
	'To close nominal expense accounts'		

```
6/3/2024    Income Summary                          0.00
              Miscellaneous Expense                             0.00
               'To close nominal expense accounts'

6/3/2024    Income Summary                          0.00
              Supplies Expense                                  0.00
               'To close nominal expense accounts'

6/3/2024    Income Summary                          0.00
              Depreciation Expense - Building                   0.00
               'To close nominal expense accounts'

6/3/2024    Income Summary                          0.00
              Depreciation Expense - Equipment                  0.00
               'To close nominal expense accounts'

6/3/2024    Nena, Capital                           65000.00
              Income Summary                                    65000.00
               'To close profit/loss'

6/3/2024    Nena, Capital                           10000.00
              Nena, Withdrawals                                 10000.00
               'To close drawings'

                                 General Ledger
                            Income Summary
6/3/2024    To close nominal income accounts                    5000.00
6/3/2024    To close nominal income accounts        0.00
6/3/2024    To close nominal income accounts        0.00
6/3/2024    To close nominal expense accounts                   0.00
6/3/2024    To close nominal expense accounts                   0.00
6/3/2024    To close nominal expense accounts       30000.00
6/3/2024    To close nominal expense accounts       0.00
6/3/2024    To close nominal expense accounts       0.00
6/3/2024    To close nominal expense accounts       0.00
6/3/2024    To close nominal expense accounts       0.00
6/3/2024    To close nominal expense accounts       0.00
6/3/2024    To close profit/loss                                65000.00

-40000.00
```

To be implemented accounting cycle function (Currently in a separate file)

Bibliography:

Alusen, Ma. L., & Javier, J. N. (2018). Accounting Practices of Sari-Sari Stores in Brgy. Makiling, Calamba, Laguna: Basis for Community Extension Program of the College of Business and Accountancy. PU-Laguna Journal of Multidisciplinary Research, 5. https://lpulaguna.edu.ph/wp-content/uploads/2019/01/1.-Accounting-Practices-of-Sari-Sari-Stores-in-Brgy.-Makiling-Calamba-Laguna-Basis-for-Community-Extension-Program-of-the-College-of-Business-and-Accountancy.pdf